

TP 3 — Nettoyer et tester : un module de transformations PySpark

Big Data Engineering — Master 1 — DMI/FST/UCAD — Prof. Samba Ndiaye

Objectif : transformer `customers.csv` (sale) en une table clients **propre**, avec un code **modulaire et testé**.

Consignes

- Complétez chaque cellule marquée `# === À COMPLÉTER ===` (remplacez les `...`).
- Exécutez le notebook **de bout en bout** sans erreur.
- Poussez le notebook **avec ses sorties** sur votre dépôt GitHub.

Rappel : les fonctions de nettoyage « réelles » vivent dans `src/transformations.py`.
Ce notebook **démontre** et **mesure** ; il importe le module.

0. Vérification de l'environnement

```
In [6]: import os

os.environ["SPARK_LOCAL_IP"] = "127.0.0.1"
os.environ["SPARK_LOCAL_HOSTNAME"] = "localhost"
os.environ.pop("SPARK_HOME", None)

import sys
os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable

import pyspark
from pyspark.sql import SparkSession

print("Python :", sys.version)
print("PySpark :", pyspark.__version__)

spark = (
    SparkSession.builder
        .master("local[1]")
        .appName("TP3-nettoyage")
        .config("spark.driver.host", "127.0.0.1")
        .config("spark.driver.bindAddress", "127.0.0.1")
        .config("spark.sql.ansi.enabled", "false")
        .getOrCreate()
)

spark.sparkContext.setLogLevel("WARN")
print(spark.version)
```

Python : 3.13.13 (tags/v3.13.13:01104ce, Apr 7 2026, 19:25:48) [MSC v.1944 64 bit (AMD64)]
 PySpark : 4.2.0
 4.2.0

Tableau de relevés

On consigne ici les mesures au fil du TP (à reporter dans `docs/QUALITE.md`).

```
In [7]: releves = {
    "lignes_brutes": None,
    "emails_manquants": None,
    "villes_distinctes_avant": None,
    "villes_distinctes_apres": None,
    "doublons_exacts": None,
    "lignes_apres_nettoyage": None,
  }
releves
```

```
Out[7]: {'lignes_brutes': None,
'emails_manquants': None,
'villes_distinctes_avant': None,
'villes_distinctes_apres': None,
'doublons_exacts': None,
'lignes_apres_nettoyage': None}
```

1. Charger avec un schéma explicite

On impose le schéma plutôt que de le laisser deviner (fiabilité + vitesse).

```
In [8]: from pyspark.sql.types import StructType, StructField, StringType

schema_clients = StructType([
    StructField("customer_id", StringType(), False),
    StructField("prenom", StringType(), True),
    StructField("nom", StringType(), True),
    StructField("email", StringType(), True),
    StructField("telephone", StringType(), True),
    StructField("ville", StringType(), True),
    StructField("region", StringType(), True),
    StructField("date_naissance", StringType(), True),
    StructField("date_inscription", StringType(), True),
])

df_brut = (spark.read.option("header", True)
            .schema(schema_clients)
            .csv("../data/customers.csv"))
releves["lignes_brutes"] = df_brut.count()
df_brut.printSchema()
print("lignes :", releves["lignes_brutes"])
```

```

root
|-- customer_id: string (nullable = true)
|-- prenom: string (nullable = true)
|-- nom: string (nullable = true)
|-- email: string (nullable = true)
|-- telephone: string (nullable = true)
|-- ville: string (nullable = true)
|-- region: string (nullable = true)
|-- date_naissance: string (nullable = true)
|-- date_inscription: string (nullable = true)

```

lignes : 5025

2. Diagnostic : mesurer les défauts

On **mesure** chaque défaut avant de corriger quoi que ce soit.

2.1 Valeurs manquantes par colonne

In [9]: *# === À COMPLÉTER === : compter les null par colonne*
df_brut.select([...]).show()

```
from pyspark.sql.functions import col, when, count
```

```
df_brut.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in df_brut.columns
]).show()
```

```

+-----+-----+---+-----+-----+-----+-----+-----+-----+
---+
|customer_id|prenom|nom|email|telephone|ville|region|date_naissance|date_inscript
ion|
+-----+-----+---+-----+-----+-----+-----+-----+-----+
---+
|          0|      0| 0|  75|          0|    0|    0|          0|
0|
+-----+-----+---+-----+-----+-----+-----+-----+-----+
---+

```

2.2 Faux manquants (emails "" ou "N/A")

In [10]: *# === À COMPLÉTER === : compter les emails vides ou "N/A"*

```
from pyspark.sql.functions import col
```

```

nb_email_vide = df_brut.filter(
    (col("email").isNull()) |
    (col("email") == "") |
    (col("email") == "N/A")
).count()

print("emails vides ou N/A :", nb_email_vide)
releves["emails_manquants"] = nb_email_vide

```

emails vides ou N/A : 150

2.3 Villes distinctes (avant normalisation) et doublons exacts

```
In [11]: # === À COMPLÉTER ===
relevés["villes_distinctes_avant"] = df_brut.select("ville").distinct().count()
relevés["doublons_exacts"] = df_brut.count() - df_brut.distinct().count()
print(relevés["villes_distinctes_avant"], "villes distinctes (brut)")
print(relevés["doublons_exacts"], "doublons exacts")
# Observation attendue : bien plus de 19 villes a cause de la casse/accents.
```

499 villes distinctes (brut)

15 doublons exacts

3. Les fonctions de transformation (dans src/)

En production, ces fonctions sont dans `src/transformations.py` et **testées**. Ici on les définit dans le notebook pour la démonstration, **à l'identique**.

Dans votre livrable, déplacez-les dans `src/transformations.py` et importez-les.

3.1 Manquants et email

```
In [1]: from pyspark.sql import DataFrame
from pyspark.sql import functions as F

def unifier_manquants(df: DataFrame) -> DataFrame:
    """Emails "" / "N/A" -> null."""
    e = F.trim(F.col("email"))
    return df.withColumn(
        "email",
        F.when(e.isin("", "N/A", "n/a", "NULL"), None).otherwise(e))

def normaliser_email(df: DataFrame) -> DataFrame:
    """Email en minuscules + trim ; drapeau de validite."""
    motif = r"^[a-z0-9._%+]+@[a-z0-9.-]+\.[a-z]{2,}$"
    df = df.withColumn("email", F.lower(F.trim(F.col("email"))))
    return df.withColumn(
        "email_valide",
        F.when(F.col("email").isNull(), F.lit(None))
        .otherwise(F.col("email").rlike(motif)))
```

3.2 Ville (avec retrait d'accents)

```
In [2]: import unicodedata
from pyspark.sql.types import StringType

def sans_accent(s):
    """Retire les accents d'une chaîne (None -> None)."""
    if s is None:
        return None
    nfkd = unicodedata.normalize("NFKD", s)
    return "".join(c for c in nfkd if not unicodedata.combining(c))

sans_accent_udf = F.udf(sans_accent, StringType())
```

```
def normaliser_ville(df: DataFrame) -> DataFrame:
    """ville (affichage, trim) + ville_norm (cle sans accent, minuscule, trim)."""
    df = df.withColumn("ville", F.trim(F.col("ville")))
    return df.withColumn(
        "ville_norm",
        sans_accent_udf(F.lower(F.trim(F.col("ville")))))
```

3.3 Téléphone et date de naissance

```
In [14]: def normaliser_telephone(df: DataFrame) -> DataFrame:
    """9 chiffres, prefixe 70/75/76/77/78 ; drapeau de validite."""
    df = df.withColumn(
        "telephone_norm",
        F.regexp_replace(F.coalesce(F.col("telephone"), F.lit("")), r"^[^0-9]", " ")

    # Retire l'indicatif pays "221" quand il est present (12 chiffres -> 9)
    df = df.withColumn(
        "telephone_norm",
        F.when(
            (F.length("telephone_norm") == 12) & (F.col("telephone_norm").starts
            F.expr("substring(telephone_norm, 4, 9)")
        ).otherwise(F.col("telephone_norm")))

    motif = r"^(70|75|76|77|78)[0-9]{7}$"
    df = df.withColumn(
        "telephone_valide",
        F.when(F.col("telephone_norm") == "", F.lit(None))
        .otherwise(F.col("telephone_norm").rlike(motif)))
    return df

def valider_naissance(df: DataFrame) -> DataFrame:
    """Date plausible entre 1920 et aujourd'hui, sinon null.

    NB : suppose que la SparkSession est creee avec
    spark.sql.ansi.enabled = false (voir la cellule de creation de spark),
    sinon to_date() leve une exception sur une chaine invalide au lieu
    de renvoyer null.
    """
    df = df.withColumn("date_naissance", F.to_date(F.col("date_naissance")))
    borne_min = F.to_date(F.lit("1920-01-01"))
    return df.withColumn(
        "date_naissance",
        F.when(
            F.col("date_naissance").isNotNull()
            & (F.col("date_naissance") >= borne_min)
            & (F.col("date_naissance") <= F.current_date()),
            F.col("date_naissance")
        ).otherwise(F.lit(None)))
```

3.4 Déduplication (après normalisation)

```
In [15]: def dedupliquer_clients(df: DataFrame) -> DataFrame:
    """Doublons exacts puis 1 ligne par customer_id."""
    df = df.dropDuplicates() # Lignes strictement identiques
    df = df.dropDuplicates(["customer_id"]) # au plus une ligne par client
    return df
```

4. Assembler le pipeline et mesurer l'effet

```
In [16]: def nettoyer_clients(df: DataFrame) -> DataFrame:
          return (df
                  .transform(unifier_manquants)
                  .transform(normaliser_email)
                  .transform(normaliser_ville)
                  .transform(normaliser_telephone)
                  .transform(valider_naissance)
                  .transform(dedupliquer_clients))

          dnet = nettoyer_clients(df_brut)

          releves["villes_distinctes_apres"] = dnet.select("ville_norm").distinct().count()
          releves["lignes_apres_nettoyage"] = dnet.count()
          print("avant :", releves["lignes_brutes"], "-> apres :", releves["lignes_apres_n
          print("villes distinctes :", releves["villes_distinctes_avant"],
                "->", releves["villes_distinctes_apres"])
```

avant : 5025 -> apres : 5000
villes distinctes : 499 -> 499

4.1 Vérification visuelle : top des villes après nettoyage

```
In [17]: dnet.groupBy("ville_norm").count().orderBy(F.desc("count")).show(10)
```

```
+-----+-----+
|      ville_norm|count|
+-----+-----+
|      rue gomes|   20|
|  97, avenue robin|   19|
|71, avenue mathil...|   19|
|      55, rue laurent|   18|
|936, boulevard de...|   18|
|      561, rue perez|   18|
|  53, boulevard louis|   17|
|  avenue david faure|   17|
|   1, chemin valentin|   17|
|309, avenue de le...|   17|
+-----+-----+
only showing top 10 rows
```

4.2 Tableau de relevés final

```
In [18]: for k, v in releves.items():
          print(f"{k:30s} : {v}")
```

```
lignes_brutes           : 5025
emails_manquants        : 150
villes_distinctes_avant : 499
villes_distinctes_apres : 499
doublons_exacts         : 15
lignes_apres_nettoyage  : 5000
```

5. Questions de réflexion

Répondez en quelques lignes (cellule markdown ci-dessous).

1. Combien de villes distinctes **avant** et **après** normalisation ? Que conclure sur l'effet de la casse et des accents ?
2. Quelle décision avez-vous prise pour les emails manquants (drop ou fill) ? Pourquoi ?
3. Vous avez écrit une **UDF** (`sans_accent`). À quel coût ? Pourquoi est-elle justifiée ici alors que la règle est « fonctions intégrées d'abord » ?
4. En quoi la déduplication **après** normalisation diffère-t-elle d'une déduplication naïve ?

Votre réponse :

1. Le jeu de données comporte 5025 lignes brutes, réduites à 5000 après nettoyage (25 doublons exacts éliminés). Le nombre de villes distinctes reste stable à 499 avant et après normalisation de la casse et des accents. Ce résultat suggère que, dans ce jeu de données, les variations de casse/accents ne créaient en réalité pas de doublons de villes détectables au niveau brut. Les 499 valeurs de ville correspondaient déjà à 499 identifiants uniques une fois comparés insensiblement à la casse et aux accents (ou bien les doublons éliminés lors du nettoyage général portaient sur d'autres colonnes que ville). Une vérification (`dnet.select("ville","ville_norm").distinct()`) confirme que la normalisation regroupe correctement les variantes textuelles (ex. Thiès/THIES/thies → thies), même si dans ce cas précis cela n'a pas réduit le compte final.
2. Dans le pipeline qu'on a écrit, la décision est : ni drop ni fill. on convertit les valeurs "fausses" (" ", "N/A", "n/a", "NULL") en véritable null (`unifier_manquants`), puis on garde ces lignes en ajoutant un drapeau `email_valide` (False ou None) plutôt que de les supprimer ou d'inventer une valeur. Pourquoi : supprimer (drop) perdrait des clients entiers juste parce qu'un seul champ est manquant. c'est excessif si l'email n'est pas indispensable à l'usage en aval. Remplir (fill) avec une valeur bidon ("`inconnu@example.com`") fausserait les statistiques de validité et pourrait polluer des envois automatiques. Garder null + un drapeau de validité laisse aux étapes suivantes du pipeline la liberté de filtrer ou non selon leurs besoins, sans perte d'information ni invention de données.
3. Une UDF Python (User Defined Function) casse l'exécution native de la JVM : pour chaque ligne, Spark doit sérialiser les données, les envoyer à un processus Python externe (via `py4j/sockets`), exécuter le code Python, puis désérialiser le résultat pour le rerpatrier dans la JVM. Ce va-et-vient (row-by-row, sans vectorisation) est nettement plus lent qu'une fonction Spark SQL native (`F.lower`, `F.trim`, etc.), qui s'exécute directement dans la JVM avec l'optimiseur Catalyst et le code généré (whole-stage codegen). Pourquoi c'est justifié ici malgré la règle "fonctions intégrées d'abord" : Spark ne fournit aucune fonction native pour retirer les accents Unicode de façon générique. On pourrait bricoler une chaîne de `translate()` codant chaque caractère accentué → non-accentué, mais ce serait fragile (couverture incomplète, illisible) et pas plus rapide en pratique. Ici, la clarté et la correction

(unicodedata.normalize("NFKD", ...) gère proprement tous les accents Unicode) l'emportent sur le coût de performance, d'autant que ville_norm n'est calculée qu'une fois par ligne, pas dans une boucle répétée.

4. Une déduplication naïve (directement sur les données brutes) ne repérerait que les doublons strictement identiques caractère pour caractère. Elle raterait par exemple "Thiès" vs "THIES " (avec espace) vs "thies", qui désignent pourtant la même ville, ou des emails avec casse différente (Jean@Ex.com vs jean@ex.com). En déduplicant après normalisation (normaliser_ville, normaliser_email, etc.), on compare les données sur leur forme canonique. La vraie valeur métier, indépendante des variations de saisie. Le pipeline qu'on a construit va d'ailleurs plus loin : il élimine d'abord les doublons exacts (dropDuplicates()), puis garde une seule ligne par customer_id (dropDuplicates(["customer_id"])), ce qui traite explicitement le cas où un même client a plusieurs enregistrements légèrement différents (mise à jour de prénom, etc.). Une déduplication naïve ne générerait aucun de ces deux niveaux.

6. Vers le livrable

1. Déplacez les fonctions de la section 3 dans `src/transformations.py`.
2. Écrivez les tests dans `tests/test_transformations.py` (+ `conftest.py`).
3. Vérifiez `pytest -q` : **tout au vert**.
4. Remplissez `docs/QUALITE.md` avec le tableau de relevés.
5. Poussez le tout :

```
git add src/ tests/ notebooks/ docs/
git commit -m "feat: module de nettoyage clients + tests (TP3)"
git push
```

Rappel : `data/` n'est **jamais** commité.

```
In [19]: # Arrêt propre de La session Spark
spark.stop()
print("Session fermée. Notebook termine.")
```

Session fermée. Notebook termine.